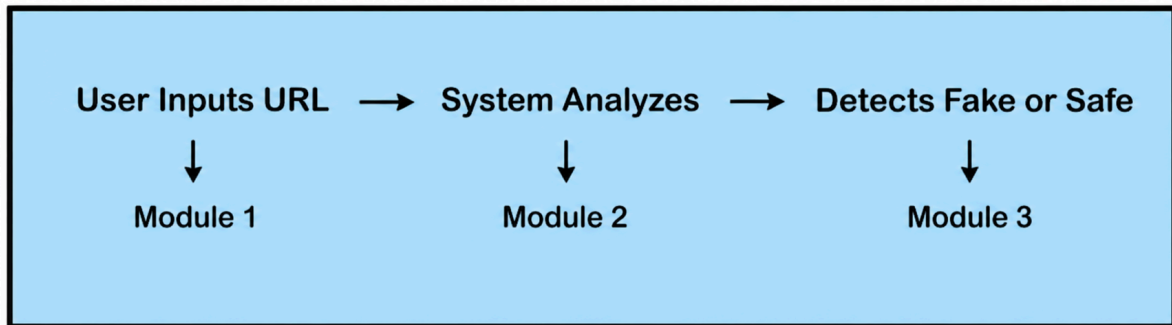


AI-Based Fake URL Detection System



1. Problem Statement

With the rapid growth of the internet, cyber attacks such as phishing have become very common. Fake websites are designed to look like real websites to trick users into entering sensitive information such as passwords, bank details, and personal data.

Many users are unable to identify these fake URLs, which leads to financial loss and data theft. Therefore, there is a need for a system that can detect suspicious URLs and warn users before they access them.

2. Why This Problem?

Phishing attacks are increasing day by day and affect:

- Students
- Online shoppers
- Bank users
- Companies

Users often cannot differentiate between real and fake URLs. This creates serious security risks.

3. Use Case (Real-World Scenario)

A user receives a link:

<https://amaz0n-login-security.xyz>

The user thinks it is Amazon and tries to log in.

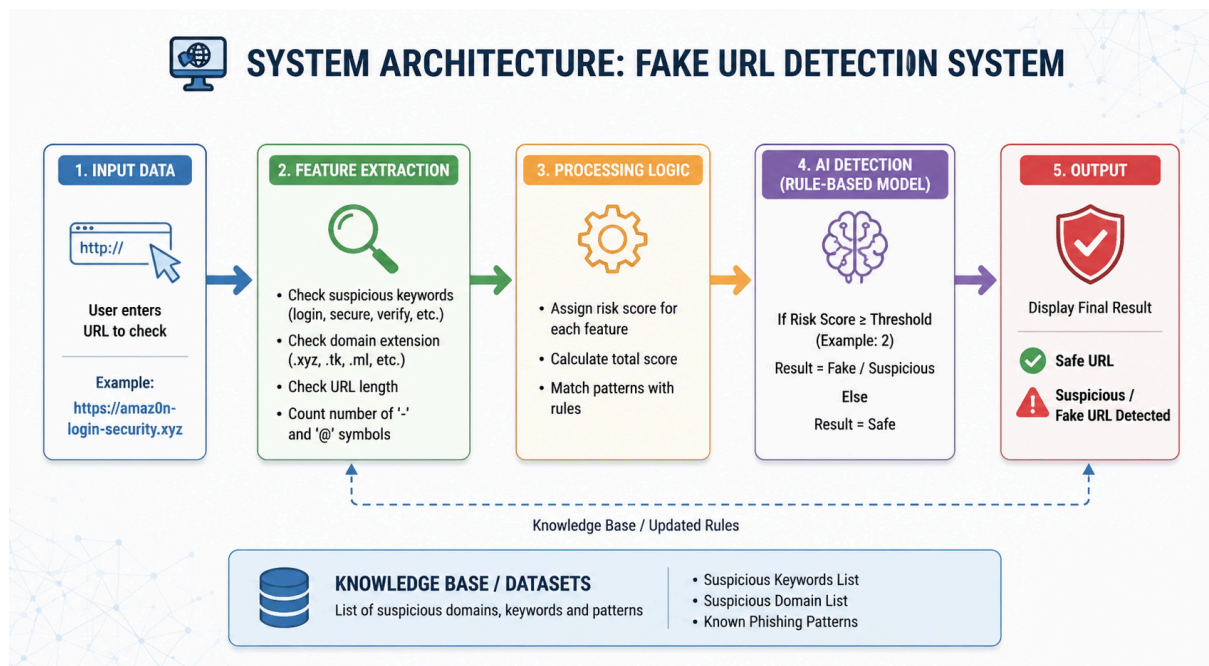
Our system analyzes the URL and detects suspicious patterns such as:

- unusual domain (.xyz)
- suspicious keywords (login, security)

It warns the user:

⚠ Suspicious / Fake URL Detected

4. How It Works (Architecture / Logic)



Step-by-step process:

1. User enters a URL
2. System converts it to lowercase
3. Checks for suspicious words
4. Counts risk score
5. If score $\geq 2 \rightarrow$ Fake URL
6. Else $\rightarrow$ Safe URL

5. AI Integration

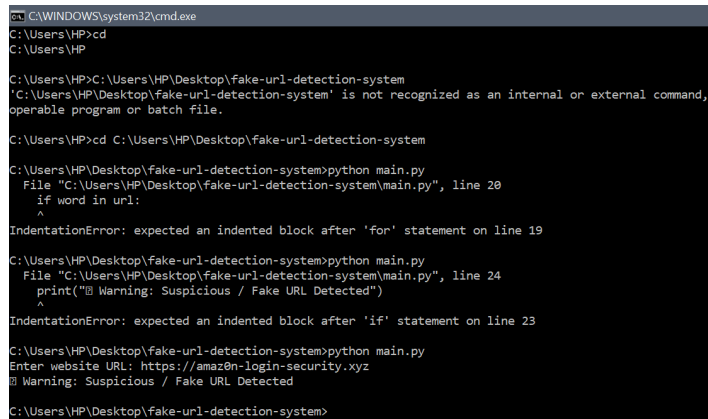
This project uses basic AI logic (rule-based detection).

AI helps in:

- identifying suspicious patterns
- detecting phishing indicators
- improving decision making

In future, machine learning models can be used for more accurate detection.

6. Prototype Implementation



```
C:\WINDOWS\system32\cmd.exe
C:\Users\HP>cd
C:\Users\HP
C:\Users\HP>cd C:\Users\HP\Desktop\fake-url-detection-system
'C:\Users\HP\Desktop\fake-url-detection-system' is not recognized as an internal or external command,
operable program or batch file.
C:\Users\HP>cd C:\Users\HP\Desktop\fake-url-detection-system
C:\Users\HP\Desktop\fake-url-detection-system>python main.py
File "C:\Users\HP\Desktop\fake-url-detection-system\main.py", line 20
    if word in url:
    ^
IndentationError: expected an indented block after 'for' statement on line 19
C:\Users\HP\Desktop\fake-url-detection-system>python main.py
File "C:\Users\HP\Desktop\fake-url-detection-system\main.py", line 24
    print("\n Warning: Suspicious / Fake URL Detected")
    ^
IndentationError: expected an indented block after 'if' statement on line 23
C:\Users\HP\Desktop\fake-url-detection-system>python main.py
Enter website URL: https://amazon-login-security.xyz
\n Warning: Suspicious / Fake URL Detected
C:\Users\HP\Desktop\fake-url-detection-system>
```

The system is implemented using Python.

It uses:

- keyword matching
- pattern detection
- conditional logic

The program takes user input and outputs whether the URL is safe or suspicious.

7. Importance & Impact

This system helps:

- prevent phishing attacks
- protect user data

- increase cybersecurity awareness
 - reduce financial fraud
-

TOOL 2: Processing & Risk Scoring System

1. Problem Statement

Simply detecting suspicious features is not enough; the system must evaluate the level of risk associated with a URL. Without proper scoring, it becomes difficult to classify whether a URL is safe or malicious. A structured scoring mechanism is required to quantify risk and improve decision-making.

2. Use Case

Cybersecurity systems, Browser extensions, Email filters
Organizations & security analysts

3. How It Works

The system assigns a score to each suspicious feature detected in the URL. For example, each keyword or suspicious domain increases the risk score. The total score is calculated and used as input for the final classification.

4. AI Integration

Uses:

- Rule-based scoring system
- Simple heuristic analysis

5. Prototype Code

TOOL1.ipynb

In [1]:

```
url = "https://amaz0n-login-security.xyz+  
suspicious_words = ["login", "secure", "verify",  
"update", "bank", "free", "bonus", "offer"]  
suspicious_domains = [".xyz", ".tk", ".ml"]  
  
url_lower = url.lower()  
found_words = [word for word in suspicious_words  
if word in url_lower]  
domain_flag = "Safe Domain"  
for domain in suspicious_domains:  
    if domain in url_lower:  
        domain_flag = "Suspicious Domain"  
        break  
  
print("URL:", url)  
print("Suspicious Keywords Found:",  
found_words if found_words else "None")  
print("Domain Status:", domain_flag)
```

URL: https://amaz0n-login-security.xyz
Suspicious Keywords Found: ['login', 'secure']
Domain Status: Suspicious Domain

In [3]:

```
if score >= 2:  
    prediction = "Fake URL"  
    confidence = 90 + score  
else:  
    prediction = "Safe URL"  
    confidence = 80  
  
print("Prediction:", prediction)  
print("Confidence:", str(confidence) + " %")
```

Prediction: Fake URL
Confidence: 93 %

In [2]:

```
score = 0  
  
# Count suspicious keywords  
for word in found_words:  
    score += 1  
  
# Check domain  
for domain in suspicious_domains:  
    if domain in url_lower:  
        score += 1  
        break  
  
print("Calculating Risk Score...")  
print("Suspicious Score:", score)
```

Calculating Risk Score...
Suspicious Score: 3

In [4]:

```
print("==== FINAL RESULT ====")  
print("Result:", prediction)
```

==== FINAL RESULT ====
Result: Fake URL

6. Importance & Impact

This tool improves accuracy by quantifying the level of risk. It helps differentiate between mildly suspicious and highly dangerous URLs, enabling better decision-making.

7. Limitations & Improvements

Limitations

- Fixed scoring rules may not adapt to new threats
- Does not consider contextual information
- Limited flexibility

Improvements

- Adaptive scoring using machine learning
- Dynamic threshold adjustment
- Integration with real-time data

TOOL 3: AI-Based URL Classification

1. Problem Statement

Users need a clear decision on whether a URL is safe or fake. Without proper classification, users may still fall victim to phishing attacks. An AI-based system is required to provide accurate and quick predictions.

2. Use Case

Browser security tools, Email filtering systems, Mobile apps
General users & organizations

3. How It Works

The system uses the calculated risk score to classify the URL. If the score exceeds a certain threshold, the URL is labeled as "Fake"; otherwise, it is considered "Safe". The system also provides a confidence level for the prediction.

4. AI Integration

Uses:

- Rule-based AI model
- Decision threshold logic

5. Prototype Code

```
TOOL3.ipynb
```

```
In [1]:  
url = "https://amaz0n-login-security.xyz"  
...  
print("Fake URL" if score >= 2 else "Safe URL")
```

```
Out[1]:  
Fake URL
```

6. Importance & Impact

This tool provides the final decision to the user, helping prevent phishing attacks and protecting sensitive information. It enhances user awareness and strengthens cybersecurity.

7. Limitations & Improvements

Limitations

- Basic AI model (rule-based)
- Limited accuracy compared to ML models
- No real-time learning

Improvements

- Use machine learning models (Random Forest, Naive Bayes)
 - Improve accuracy with large datasets
 - Add real-time prediction systems
-

FINAL CONCLUSION

The system is divided into three tools: feature extraction, processing logic, and AI classification. Together, they provide an effective solution for detecting fake URLs. Although the current system uses basic AI logic, it demonstrates how cybersecurity tools can be developed using Python. Future improvements can enhance accuracy and scalability.